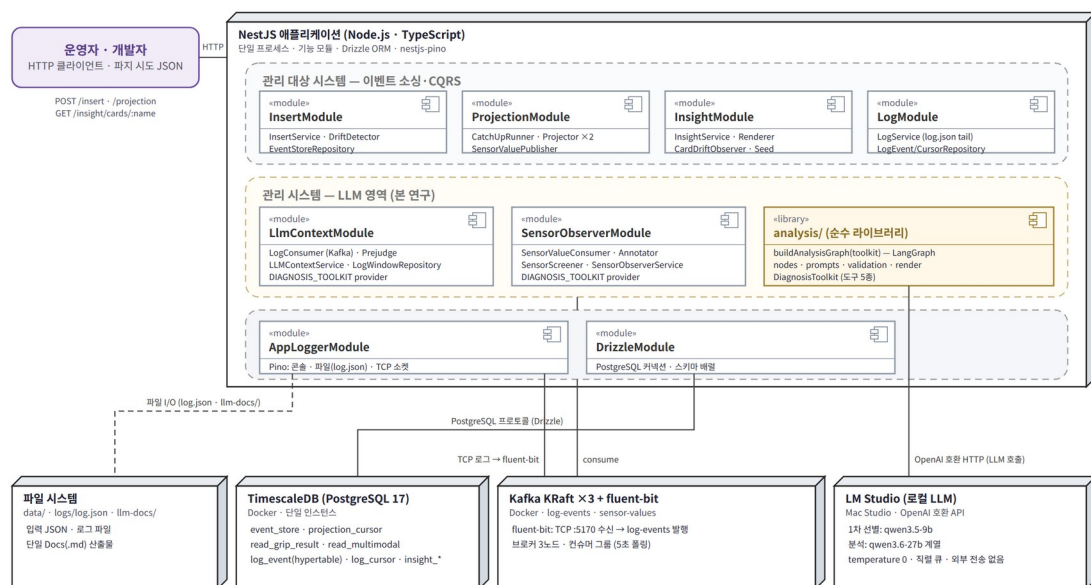


## 제 3 장 Self-Adaptive CQRS 설계 및 구현

### 3.1 제안 시스템 개요 및 설계 원칙

본 장에서는 제 2 장에서 확인한 연구 공백, 즉 Read Model 의 설계 의사결정이 개발자의 수작업에 의존하는 문제를 완화하기 위해 제안하는 Self-Adaptive CQRS 아키텍처의 설계와 구현을 기술한다. 제안 시스템은 기존의 이벤트 소싱·CQRS 구조를 관리 대상 시스템(managed system)으로 두고, 그 위에 LLM 기반의 분석 영역을 관리 시스템(managing system)으로 결합하는 구조를 갖는다[12]. 관리 대상 시스템의 코드와 데이터 모델은 변경하지 않으며, 관리 시스템은 관리 대상 시스템이 운영 중에 방출하는 로그와 데이터를 관측하여 Read Model의 재생성이 필요한 상황을 감지하고, 그 대응을 단일 문서(이하 Docs)로 산출한다. 그림은 제안 시스템의 구성도로, NestJS 단일 프로세스 안에 배치된 기능 모듈과 외부 인프라(TimescaleDB, Kafka, 로컬 LLM 서버, 파일 시스템)의 관계를 나타낸 것이다.



관리 대상 시스템(상단 모듈)은 변경 없이 로그·데이터로만 관리 시스템(LLM 영역)과 연결되며, 모든 LLM 호출은 로컬 LM Studio로만 향한다

그림 3-1. Self-Adaptive CQRS 시스템 구성도

제안 시스템은 네 개의 영역으로 구성된다. 첫 번째는 기존의 이벤트 소싱·CQRS 영역으로, 쓰기 측의 적재 서비스가 Physical AI 파지 시도 데이터를 검증하여 Event Store에 불변 이벤

트로 기록하고 읽기 측의 프로젝션 실행기가 이벤트를 목적별 Read Model 에 반영한다. 두 번째는 관측 백본으로, 모든 구성 요소가 남기는 구조화된 로그가 메시지 브로커와 로그 데이터베이스로 동시에 전달되며, 프로젝션이 반영한 센서 값은 오류가 아닌 데이터이므로 별도의 토픽으로 발행된다. 세 번째는 본 연구가 추가한 LLM 영역으로, 유입되는 이상 신호를 결정론적 게이트와 소형 LLM 이 1차로 선별하고 선별된 신호에 대해서만 분석 그래프가 근본원인 분석, 의사결정, 산출물 생성을 거쳐 Docs 를 생성한다. 네 번째는 LLM 컨텍스트 소스로, 개발자의 구조화된 로그를 축적한 Log DB 와 시스템이 보유한 이벤트·Read Model 의 스키마와 의미를 카드 형태로 관리하는 Insight Read DB 가 분석 그래프에 컨텍스트를 제공한다. 생성된 Docs 는 사람의 승인을 거쳐 관리 대상 시스템에 적용되고, 이때 신규 Read Model 의 카드가 Insight Read DB 에 함께 등록되어 다음 분석의 컨텍스트로 순환한다. 이 순환이 제안 시스템에서 자가 적응을 구성하는 폐루프이다.

제안 시스템의 설계는 다섯 가지 원칙을 따른다. 첫째는 판단과 실행의 분리이다. LLM 은 어떤 이상이 왜 발생하였는지, 어떤 조치가 필요한지, 신규 Read Model 의 컬럼을 어떻게 구성할지와 같이 도메인 이해와 생성 능력이 필요한 판단만 담당하고, 신호의 선별 게이트, 산출물의 검증, SQL 과 코드의 렌더링, 문서의 조립은 결정론적 코드가 담당한다. 이는 LLM 산출물의 불확실성[24]과 소형 로컬 모델을 사용해야 하는 배포 환경의 제약[17]을 함께 고려한 결정이다. 둘째는 감지는 열린 집합으로, 행동은 닫힌 집합으로 다루는 것이다. 근본원인 분석이 판정하는 이상 유형은 미리 열거하지 않은 유형의 명명을 허용하는 열린 집합(open-set)이지만, 이후의 조치는 API 버전 전환, 권고 문서, 신규 Read Model, 데이터 품질 권고의 네 가지로 고정된 닫힌 열거형이다. 이는 적응 공간이 설계 시점에 닫혀 있는 전통적 자가 적응[10, 30]과 적응 행동을 실행 시점에 생성하는 LLM 기반 접근[24] 사이의 절충으로, 미지의 신호가 미지의 행동으로 이어지지 않도록 하는 경계이다. 셋째는 단일 산출물의 완전성이다. Docs 는 권고 문서, Read Model 생성 SQL, API Versioning 의 세 요소를 항상 함께 담으며, 신규 Read Model 이 선택되면 권고와 버전 전환이 반드시 동반되도록 불변식으로 강제하고, 근거가 부족한 요소는 생략하지 않고 근거 부족을 명시하는 센티넬로 채운다. 넷째는 2 단 선별에 의한 비용 통제이다. 모든 배치에 실행되는 1차 선별은 결정론적 게이트와 소형 LLM 으로 구성하

고, 추론과 도구 호출을 반복하는 고비용의 분석 그래프는 선별을 통과한 소수의 신호에 대해서만 실행한다. 다섯째는 human-in-the-loop 와 루프의 안정성이다. DDL 실행과 API 컷오버는 사람의 승인 이후에만 이루어지며, 분석 파이프라인의 자기 로그는 이상 신호로 재유입되지 않도록 정보 수준으로만 기록한다. 표는 각 원칙이 구현된 위치를 정리한 것이다.

| 설계 원칙                 | 내용                                                | 구현 위치                               |
|-----------------------|---------------------------------------------------|-------------------------------------|
| 판단과 실행의 분리            | LLM 은 진단·선택·설계만, 게이트·검증·렌더링·조립은 결정론 코드            | 프리게이트, 검증 관문, 렌더러 (3.4 절, 3.5 절)    |
| 열린 감지·닫힌 행동           | 이상 유형은 open-set 문자열, 조치는 4 종 닫힌 열거형               | 근본원인·의사결정 출력 스키마 (3.5 절)            |
| 단일 산출물의 완전성           | 3 요소 항상 동반, 신규 Read Model ⇒ 권고 + 버전 전환, 근거 부족 센티넬 | 의사결정 불변식, 조립 무결성 게이트 (3.4 절, 3.5 절) |
| 2 단 선별 비용 통제          | 결정론 게이트 + 소형 LLM 선별, 트립 시에만 분석 그래프                | 로그 레인 선판단, 센서 레인 스칼라 (3.5 절)        |
| Human-in-the-loop·안정성 | 승인 후 적용, 자기 로그 정보 수준 제한, 로컬 LLM 구동                | Docs 실행 게이트, 로깅 규약 (3.4.4 항, 3.6 절) |

## 3.2 Event Store 와 목적별 Read Model

관리 대상 시스템은 Physical AI 도메인의 로봇 파지 시도(grip attempt) 데이터를 이벤트 생성·CQRS 구조로 관리한다. 데이터는 로봇이 물건을 파지하는 한 번의 시도를 하나의 JSON 파일로 기록한 것으로, 2D·3D 이미지와 비디오의 파일명, 카메라 내부 파라미터, 2D·3D 파지 포즈, 로봇의 회전 행렬과 이동 벡터(robot TF), 대상 객체와 사람의 파지 주석, 파지 성공 여부의 11 개 최상위 키로 구성된다. 파일명은 카테고리, 카메라 코드, 객체명, 장면 번호, 시도 번호, 촬영 일자를 담는 규약을 따르며, 적재 서비스는 여기서 장면 식별 키(scene key)와 시도 번호를 추출한다. 하나의 장면은 하나의 이벤트 스트림(grip-attempt:<scene key>)에 대응하고, 각 시도는 스트림 내에서 시도 번호로 구분되는 하나의 GripAttemptRecorded 이벤트가 된다.

적재 서비스는 파일마다 파일명 파싱, JSON 파싱, 스키마 드리프트 감지, 스키마 검증, 이벤트 추가의 순서로 동작한다. 스키마 검증은 zod 스키마로 수행되며 스키마에 정의되지 않은 키는 검증 과정에서 제거되므로, 새로운 센서가 추가되어 스키마가 알지 못하는 키가 유입되면 그 값은 Event Store 에 기록되지 않고 유실된다. 이를 관측하기 위해 검증 이전의 원본 JSON 에

서 알려진 키 집합에 속하지 않는 최상위 키를 탐지하고, 배치 종료 시 신규 키의 목록과 값의 표본을 담은 경고 로그(payload.schema.drift)를 남긴다. 이 로그가 제 1 장에서 문제로 정의한 “스키마에 정의되지 않은 필드의 유입”을 시스템이 스스로 관측하는 첫 번째 신호이다. 이벤트 추가는 스트림과 시도 번호의 고유 제약에 대해 충돌 시 무시하는 방식으로 수행되어 재적재에 대해 먹등하다. 표는 Event Store 의 스키마를 정리한 것이다.

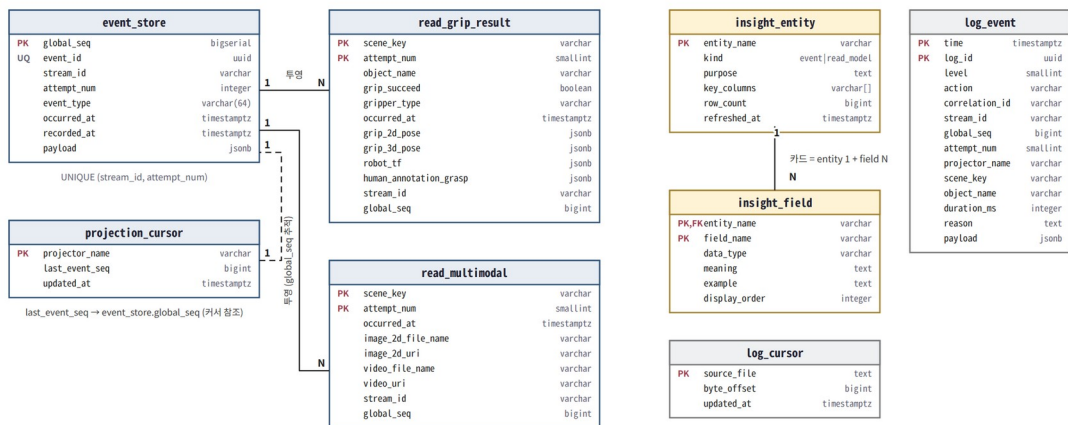
| 컬럼                       | 자료형              | 의미                                               |
|--------------------------|------------------|--------------------------------------------------|
| global_seq               | bigserial (PK)   | 전역 시퀀스, 프로젝트 커서의 기준                              |
| event_id                 | uuid (unique)    | 이벤트 식별자                                          |
| stream_id, attempt_num   | varchar, integer | 스트림 식별자(grip-attempt:<scene key>)와 시도 번호 (함께 고유) |
| event_type               | varchar(64)      | 이벤트 유형<br>(GripAttemptRecorded)                  |
| occurred_at, recorded_at | timestampz       | 사건 발생 시각(촬영 일자)과 기록 시각                           |
| payload                  | jsonb            | 원본 JSON 전체                                       |

읽기 측에는 조회 목적별로 두 개의 Read Model 이 놓인다. read\_grip\_result 는 장면별 파지 결과의 조회를 목적으로 객체명, 파지 성공 여부, 그리퍼 유형, 포즈, 로봇 TF, 파지 주석을 담고, read\_multimodal 은 장면별 미디어 링크의 조회를 목적으로 2D 이미지와 비디오의 파일명 및 저장 위치를 담는다. 두 Read Model 은 모두 (scene\_key, attempt\_num)을 기본 키로 가지며 추적을 위해 원천 이벤트의 스트림 식별자와 전역 시퀀스를 보존한다. 표는 두 Read Model 의 스키마를 정리한 것이다.

| Read Model       | 목적                             | 컬럼                                                                                                                                                                                                       | 인덱스                                                              |
|------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| read_grip_result | 장면별 파지 결과 조회<br>(성공 여부·포즈·그리퍼) | scene_key,<br>attempt_num,<br>object_name,<br>grip_succeed,<br>gripper_type,<br>occurred_at,<br>grip_2d_pose,<br>grip_3d_pose,<br>robot_tf,<br>human_annotation_g<br>rasp, stream_id,<br>global_seq (12) | (object_name,<br>occurred_at),<br>(grip_succeed,<br>occurred_at) |
| read_multimodal  | 장면별 2D 이미지·비디오                 | scene_key,                                                                                                                                                                                               | 없음                                                               |

| Read Model | 목적    | 컬럼                                                                                                                                  | 인덱스 |
|------------|-------|-------------------------------------------------------------------------------------------------------------------------------------|-----|
|            | 링크 조회 | attempt_num,<br>occurred_at,<br>image_2d_file_name,<br>image_2d_uri,<br>video_file_name,<br>video_uri, stream_id,<br>global_seq (9) |     |

그림은 관리 대상 시스템의 Event Store·Read Model·프로젝션 커서와 관리 시스템이 사용하는 Log DB·Insight Read DB의 테이블을 함께 나타낸 데이터 모델이다. Read Model의 행은 원천 이벤트의 전역 시퀀스로 논리적으로 추적되며, 물리적 외래 키 제약은 Insight 카드의 두 테이블 사이에만 둔다.



쓰기 측 — event\_store(append-only) · projection\_cursor(프로젝터별 커서)  
 읽기 측 — read\_grip\_result · read\_multimodal: PK (scene\_key, attempt\_num), 원천 추적용 stream\_id · global\_seq 보존  
 Insight Read DB — insight\_entity · insight\_field: 이벤트 · Read Model의 스키마와 의미 카드, M-Schema로 렌더링  
 Log DB — log\_event: TimescaleDB hypertable(1일 창크), 조회 필드는 1급 컬럼 + 원본 레코드 payload · log\_cursor: 파일 tail 바이트 커서  
 관계 표기: 실선 1-N = 투영(이벤트 → Read Model 행, global\_seq로 논리적 추적) · 점선 = 커서 참조 · 물리적 FK는 insight\_field → insight\_entity만

그림 3-2. 데이터 모델 (Event Store · Read Model · Log DB · Insight Read DB)

프로젝션은 프로젝터(projector)와 캐치업 실행기(catch-up runner)로 구성된다. 프로젝터는 이벤트를 행으로 변환하는 map 과 행을 갱신하는 upsert 를 필수로, 반영된 행의 의미적 정합성을 검사하는 checkIntegrity 와 행에서 센서 값을 추출하는 toSensorValueMessages 를 선택적으로 구현하는 인터페이스이다. 캐치업 실행기는 프로젝터별 커서가 가리키는 전역 시퀀스 이후의 이벤트를 500 건 단위로 읽어 한 배치의 변환·갱신 과 커서의 전진을 하나의 트랜잭션으로 처리한다. 변환 과정에서 스키마 검증에 실패하는 poison 이벤트가 발견되면 오류 로그(projection.map.failed)를 남기고 예외를 전파하여 배

치를 롤백한다. 이는 결함 이벤트를 건너뛰어 조용히 누락시키는 대신 커서를 멈추어 문제를 드러내는 fail-fast 정책이다. 배치가 커밋된 뒤에는 트랜잭션 밖에서 두 가지 후처리가 수행된다. 하나는 정합성 규칙의 검사로, 예를 들어 read\_multimodal 은 미디어 파일명이 규약대로 파싱되는지와 파일명에 담긴 장면·시도 번호가 행의 키와 일치하는지를 검사하여 위반 시 오류 로그(projection.integrity.violation)를 남긴다. 다른 하나는 센서 값의 발행으로, read\_grip\_result 에 반영된 행의 포즈·TF·성공 여부를 스트림 식별자를 키로 하는 메시지로 변환하여 별도의 토픽에 발행한다. 발행은 실패하더라도 투영을 되돌리지 않는 최선 노력(best-effort) 방식이며, 이 값은 3.5 절의 센서 레인에서 물리적 타당성을 검사하는 입력이 된다. 그림은 적재와 프로젝션의 처리 순서, 그리고 각 단계에서 이상 신호가 로그 레일과 토픽으로 방출되는 지점을 나타낸 시퀀스 다이어그램이다.

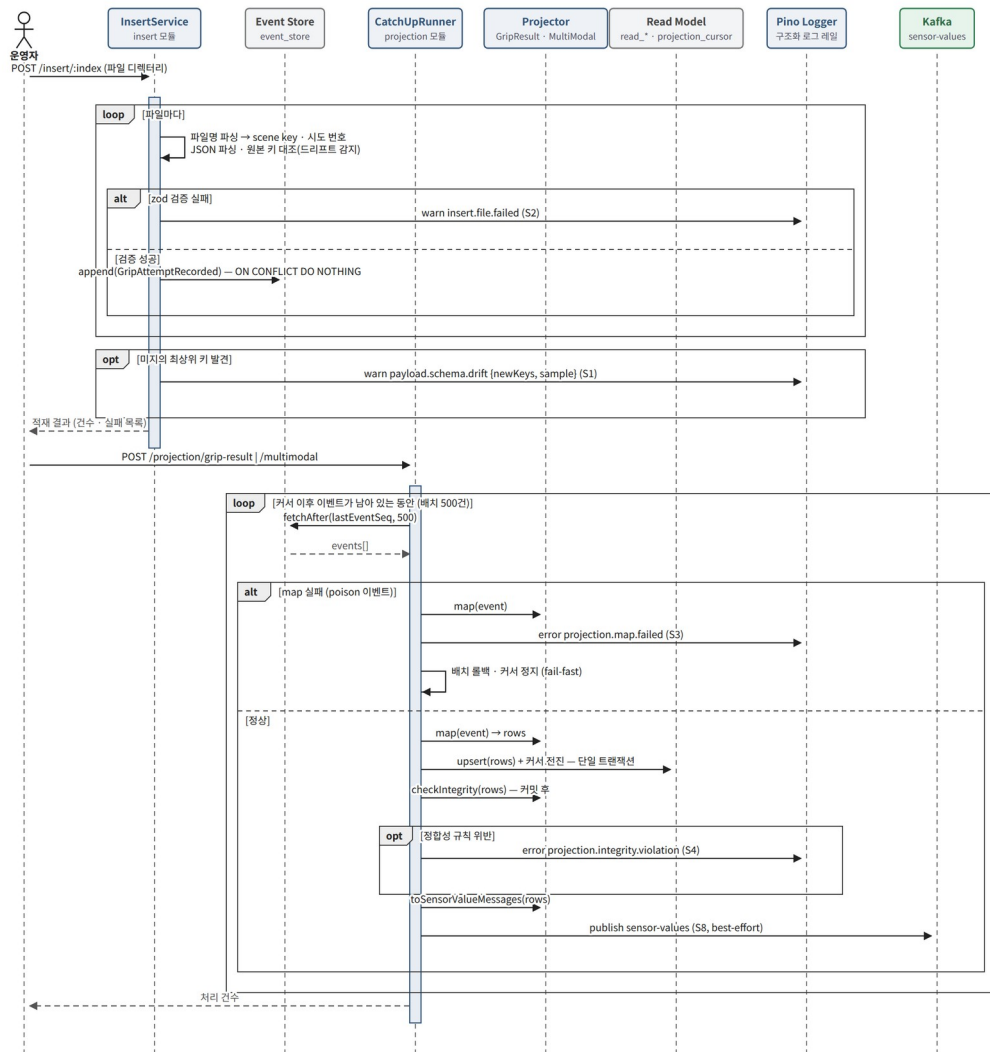


그림 3-3. 적재·프로젝션 시퀀스 다이어그램과 이상 신호 방출 지점

이 구조에서 주목할 점은 관리 대상 시스템의 각 단계가 자신의 실패와 이상을 구조화된 로그와 메시지로 외부화하도록 설계되었다는 것이다. 관리 대상 시스템은 관리 시스템을 위한 별도의 인터페이스를 갖지 않으며, 로그와 데이터라는 통상적인 관측 수단만으로 관리 시스템과 연결된다. 이 신호들이 다음 절에서 설명하는 LLM 컨텍스트의 원천이 된다.

### 3.3 LLM 컨텍스트 모델

LLM은 대상 시스템의 상태, 스키마 및 이력을 학습 과정에서 알 수 없으므로 판단에 필요한 정보를 실행 시점에 컨텍스트로 제공받아야 한다(2.3.1 항). 제안 시스템의 컨텍스트는 두 소스로부터 구성된다. 개발자 Logging 기반 컨텍스트는 “시스템에서 무슨 일이 일어났는가”를,

Insight Read DB 기반 컨텍스트는 “시스템이 무엇을 보유하고 있는가”를 담당한다. 전자가 이상의 존재와 맥락을 알려준다면, 후자는 그 이상에 대응하여 어떤 Read Model 을 어떤 컬럼으로 설계할지 판단할 때 대조할 스키마 지식을 제공한다.

### 3.3.1 개발자 Logging 기반 컨텍스트

개발자 Logging 을 컨텍스트로 활용하려면 로그가 LLM 이 해석할 수 있을 만큼 구조화되어야 한다. 이를 위해 제안 시스템은 네 가지 로깅 규약을 정의한다. 첫째, 모든 로그는 JSON 레코드로 기록되며 insert.file.failed, projection.map.failed, insight.card.miss 와 같이 영역과 사건을 연결한 행위명(action) 사건을 공유한다. 둘째, 하나의 요청에서 발생한 모든 로그는 요청 헤더 또는 자동 생성된 상관관계 식별자(correlation ID)를 공유하며, 식별자는 미들웨어를 통해 서비스 계층의 로거에 전파되어 요청의 수신부터 완료까지의 궤적을 하나의 식별자로 추적할 수 있다. 셋째, 이상 신호는 모두 경고(warn, 40) 이상의 수준으로 기록한다. 이 규약 하나로 로그 레인의 모든 이상 신호가 수렴하므로, 새로운 탐지기를 추가할 때 파이프라인을 수정할 필요 없이 경고 로그 한 줄을 남기는 것으로 충분하다. 넷째, 분석 파이프라인 자신의 로그는 정보(info) 수준으로만 기록한다. 파이프라인의 로그도 같은 경로로 재유입되므로 분석 결과를 경고 수준으로 기록하면 자기 로그가 이상 탐지를 다시 유발하는 발진(oscillation)이 생기기 때문이다. 표는 관리 대상 시스템이 방출하는 이상 신호를 정리한 것이다.

| 신호               | 행위명                            | 수준    | 발생 지점       | 의미                                           |
|------------------|--------------------------------|-------|-------------|----------------------------------------------|
| S1 페이로드 스키마 드리프트 | payload.schema.drift           | warn  | 적재 서비스      | 스키마가 알지 못하는 신규 키 유입, 적재 시 유실 (Read Model 후보) |
| S2 적재 거절         | insert.file.failed             | warn  | 적재 서비스      | 스키마 검증 실패, 이벤트 미기록 (원천 데이터 결함)               |
| S3 투영 변환 실패      | projection.map.failed          | error | 프로젝터        | poison 이벤트, 배치 롤백·커서 정지                      |
| S4 투영 정합성 위반     | projection.integrity.violation | error | 캐치업 실행기     | 구조는 유효하나 의미 규칙 위반                            |
| S5 카드 드리프트       | insight.card.drift             | warn  | 카드 드리프트 관찰자 | 카드가 등록되지 않은 Read Model 테                     |

| 신호                   | 행위명                    | 수준         | 발생 지점       | 의미                                           |
|----------------------|------------------------|------------|-------------|----------------------------------------------|
| S6 카드 조회 실패          | insight.card.miss      | warn       | Insight 서비스 | 이블 존재<br>존재하지 않는 카드<br>조회 (사용자 조회<br>의도의 표출) |
| S7 HTTP 이상·반<br>복 요청 | (요청 완료 로그)             | warn/error | HTTP 로거     | 4xx/5xx 응답, 동일<br>요청 반복, 시퀀스<br>이상           |
| S8 센서 값 베이스<br>라인 이탈 | (sensor-values 토<br>픽) | —          | 캐치업 실행기     | 값 자체는 오류가 아<br>니므로 로그가 아닌<br>별도 토픽으로 전달      |

이 가운데 S6 은 본 연구의 문제 정의와 직접 연결되는 신호이다. 사용자가 원하는 데이터를 기  
존 Read Model 이 제공하지 못하는 상황은 시스템 내부의 오류로 나타나지 않는다. 제안 시  
스템은 Insight Read DB 의 카드를 조회하는 경로를 조회 요구의 진입점으로 두고, 존재하지  
않는 카드에 대한 조회를 경고 로그로 기록한다. 조회 대상의 이름에는 “객체별 파지 성공률을  
한 번에 조회하고 싶다”와 같은 자연어 질의가 그대로 담기며, 같은 질의가 반복되면 단발성 오  
타가 아닌 조회 의도의 신호로 해석된다.

로그는 두 경로로 전달된다. 하나는 로거의 TCP 전송이 수집기를 거쳐 메시지 브로커의 log-  
events 토픽으로 흐르는 실시간 경로로 1 차 선별의 입력이 되고, 다른 하나는 로그 파일을 주  
기적으로 읽어 Log DB 의 log\_event 테이블에 적재하는 경로로 분석 시점에 이상의 전후 맥  
락을 조회하는 데 쓰인다. log\_event 는 시각을 기준으로 분할되는 시계열 테이블이며, 수준·  
행위명·상관관계 식별자·스트림 식별자·전역 시퀀스·장면 키 등 조회에 쓰이는 필드를 1 급 컬  
럼으로, 원본 레코드 전체를 JSON 컬럼으로 보존한다. 두 경로를 분리한 이유는 브로커가 배  
치 단위의 실시간 선별에 적합한 반면, 앵커 시각을 기준으로 전후의 로그를 정확히 조회하는  
작업에는 시각 인덱스를 갖춘 데이터베이스가 필요하기 때문이다.

분석 그래프에 주입되는 로그 컨텍스트는 이상 로그 윈도우(anomaly log window)라는 단  
위로 조립된다. 1 차 선별이 반환한 트립 상관관계 식별자로 해당 요청의 궤적을 조회하고 그  
가운데 첫 번째 경고 이상 행을 앵커(anchor)로 정한다. 앵커의 전후 20 행씩을 취한 뒤, 경고  
이상 행과 트립 식별자를 공유하는 행을 신호 행으로 보고 각 신호 행의 앞 4 행과 뒤 6 행만 남  
기는 잡음 제거를 거쳐 80 행 이내로 제한하며, 앵커 이전 1 시간의 행위명·수준별 발생 빈도를  
함께 조립한다. 각 행은 시각, 수준, 행위명, 상관관계 식별자, 스트림 식별자, 시도 번호, 전역

시퀀스, 메시지, 그리고 원본 레코드의 이유·파일명·이벤트 식별자·신규 키를 합성한 상세 열로 구성되고 앵커 행에는 트립 앵커 표시가 붙는다. 상세 열을 별도로 두는 이유는 분석 LLM 이 격리 SQL 의 조건절을 작성할 때 실재하는 이벤트 식별자와 장면 키를 인용하도록 하기 위함이다. 그림은 로그의 수집에서 1차 선별, 윈도우 조립, 분석 그래프 호출과 Docs 저장에 이르는 로그 레인의 처리 순서를 나타낸 시퀀스 다이어그램이다.

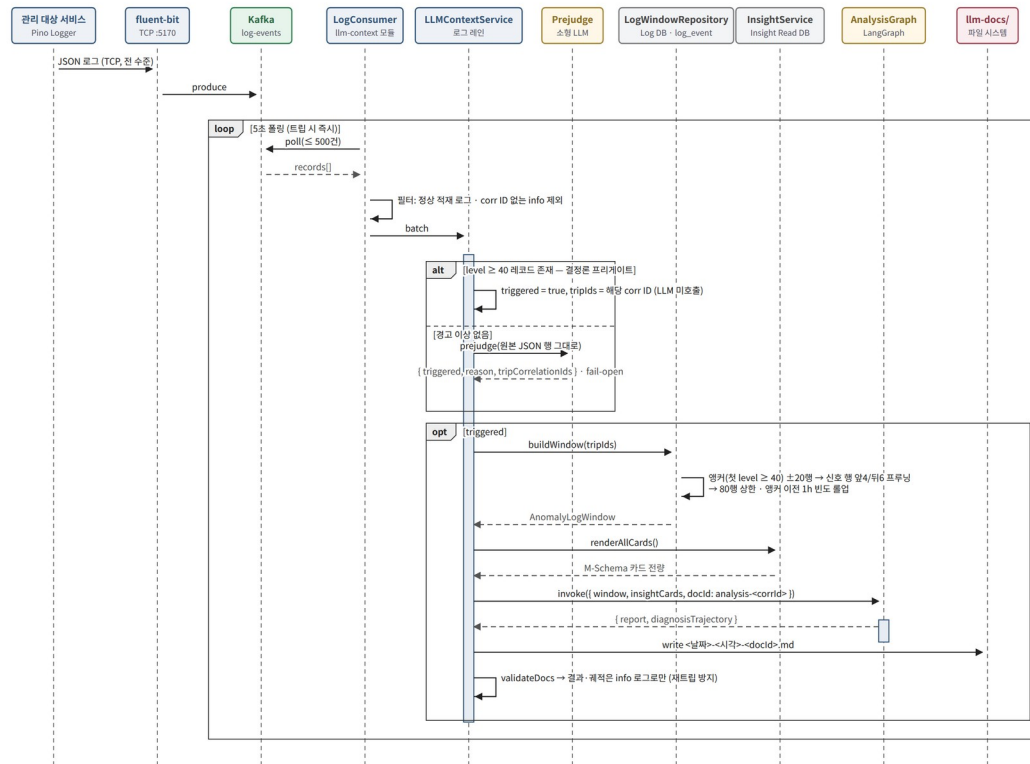


그림 3-4. 로그 레인 시퀀스 다이어그램 (수집 → 1차 선별 → 윈도우 조립 → 분석 → Docs)

### 3.3.2 Insight Read DB 기반 컨텍스트

Insight Read DB 는 시스템이 보유한 이벤트와 Read Model 의 스키마 및 의미를 LLM 이 참조할 수 있는 형태로 관리하는 메타데이터 저장소이다. LLM 의 SQL 생성 품질은 스키마 정보를 어떻게 제공하는지에 크게 좌우되며[14-17], 본 논문이 다루는 DDL 생성은 개별 테이블의 컬럼 정보를 넘어 이벤트의 의미와 기존 Read Model 의 목록·용도까지 필요로 한다.

Insight Read DB 는 이 요구를 카드(card)라는 단위로 충족한다. 카드 한 장은 하나의 엔티티를 기술하며, 이름·종류·용도·키 컬럼을 담은 insight\_entity 한 행과 각 필드의 이름·자료형·의미·예시 값·표시 순서를 담은 insight\_field 여러 행으로 구성된다. 엔티티의 종류는 현

재 조회 가능한 완성품인 Read Model 과 아직 Read Model 로 구체화되지 않은 재료인 이벤트의 두 가지이다. 초기 카탈로그에는 두 Read Model(12 필드, 9 필드)과 원천 이벤트 GripAttemptRecorded(23 필드)의 세 장이 등록되며, 이벤트 카드는 payload 의 중첩 구조를 camera\_info.camera\_name, objects[].class\_name 과 같은 경로 표기로 평탄화하여 기술하므로 LLM 은 어떤 원천 필드가 아직 어떤 Read Model 에도 반영되지 않았는지를 카드의 대조만으로 파악할 수 있다.

카드는 LLM 에 주입될 때 Text-to-SQL 연구에서 제안된 M-Schema 표현[16]으로 렌더링된다. M-Schema 는 테이블을 (필드명:자료형, 의미, 기본 키 여부, 예시 값)의 반구조화된 튜플 목록으로 기술하는 방식으로, 컬럼의 의미와 예시를 함께 담아 스키마 링크의 정확도를 높이면서도 DDL 원문보다 토큰 효율이 높다. 렌더러는 엔티티의 종류에 따라 ## ReadModel: 또는 ## Event: 제목을 붙이고 용도와 키 컬럼을 기술한 뒤 튜플 목록을 코드 블록으로 출력하며, 기본 키 여부는 키 컬럼 정의로부터 결정론적으로 판정한다. 그림은 Insight Read DB 의 카드 구조와 M-Schema 렌더링의 예를 나타낸 그림이다.



그림 3-5. Insight Read DB 카드 구조와 M-Schema 렌더링

Insight Read DB 는 컨텍스트 소스인 동시에 두 이상 신호의 발생 지점이다. 존재하지 않는 카드에 대한 조회는 카드 조회 실패 신호(S6)를 발생시키고, 카드 드리프트 관찰자는 60 초 주기로 데이터베이스의 실제 테이블 목록에서 인프라 테이블과 카드가 등록된 테이블을 제외한 나머지를 계산하여 카드 없이 존재하는 Read Model 테이블을 발견하면 테이블별 1시간의 억

제 창을 두고 카드 드리프트 신호(S5)를 발생시킨다. 후자는 사람이 신규 Read Model 을 만 들고 카드 등록을 누락하였을 때 카탈로그와 실제 스키마의 불일치를 시스템이 스스로 드러내 는 장치이다.

### 3.3.3 컨텍스트 조합 및 주입 전략

두 소스의 컨텍스트는 파이프라인이 밀어 넣는 push 방식과 진단 에이전트가 필요할 때 끌어 오는 pull 방식으로 분석 그래프에 주입된다. push 방식에서는 그래프의 호출 시점에 이상 로 그 윈도우(로그 레인) 또는 센서 이상 발견(센서 레인) 가운데 하나와 Insight Read DB 의 전 체 카드가 그래프의 상태로 전달된다. 카드를 관련성에 따라 선별하지 않고 전량 주입하는 것 은 신규 Read Model 의 설계가 기존 모델 전체와의 대조를 요구하고 카탈로그의 규모가 컨텍 스트 예산 안에 들어오기 때문이다. pull 방식에서는 근본원인을 분석하는 진단 에이전트가 다 섯 가지 도구로 Log DB, Insight Read DB, 센서 베이스라인, 소스 코드를 직접 조회한다. 표 는 진단 도구의 명세를 정리한 것이다.

| 도구                  | 역할                                                                     | 입력                                               | 통제 장치                                          |
|---------------------|------------------------------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| search_logs         | Log DB 조건 조회: 상관 관계 식별자로 요청 궤적 추적, 행위명·수준으로 반 복 패턴 확인, 오류 스택 상단 6 줄 발췌 | correlationId, action, sceneKey, minLevel, limit | 최대 40 행                                        |
| list_insight_cards  | 카탈로그의 엔티티 이름 목록                                                        | 없음                                               | —                                              |
| get_insight_card    | 카드 한 장의 M-Schema 렌더링                                                   | entityName                                       | —                                              |
| get_sensor_baseline | 센서 값의 규칙명·기대 범 위·분포 통계를 담은 베이 스라인 문서                                   | 없음                                               | —                                              |
| read_source_code    | 소스 파일을 행 번호와 함 께 열람                                                    | filePath, startLine, endLine                     | 저장소 내부 src/·dist/만 허용(default-deny), 최 대 120 행 |

모든 도구의 결과는 4,000 자에서 절단되며 절단 시 조건을 좁혀 재조회하라는 안내가 부기된 다. 이는 잘못된 증거가 아니라 과다한 증거로부터 판정 품질을 보호하는 컨텍스트 통제 장치 이다. push 는 모든 분석에 공통으로 필요한 최소 컨텍스트를 예측 가능한 비용으로 보장하고,

pull 은 오류 스택이 가리키는 소스 코드의 특정 행이나 특정 카드의 상세처럼 이상의 유형에 따라 달라지는 추가 증거를 필요한 경우에만 조회하게 하여 두 방식은 상보적으로 작동한다. 분석 그래프 내부에서는 노드마다 서로 다른 컨텍스트가 조립된다. 근본원인 분석 노드는 윈도우 또는 센서 발견을 근거 텍스트로 받고 도구로 증거를 보강한다. 의사결정 노드는 근본원인 분석의 결과에 윈도우 표와 카드를 더하여 받는데, 이는 카드 조회 실패의 반복 횟수처럼 요약된 근본원인만으로는 판단할 수 없는 사실을 원문에서 확인하기 위함이다. 생성기 노드는 여기에 기존 프로젝터·스키마·컨트롤러의 소스 코드 발췌를 추가로 받아 생성 코드가 실제 시그니처와 스타일을 따르도록 한다. 특히 신규 Read Model 이 선택된 경우, 먼저 확정된 설계(테이블명, 키, 필드 목록, DDL)가 이후의 권고 문서, 버전 전환, 데이터 품질 노드에 “확정된 설계 — 그대로 재사용하라”는 블록으로 주입된다. 이는 생성기마다 스키마를 즉흥적으로 창안하여 문서의 각 절이 서로 다른 테이블을 가리키던 자기모순을 막기 위한 장치로, 그래프의 실행 순서와 함께 3.5 절에서 다시 설명한다.

컨텍스트 주입에는 두 가지 위생 규칙이 적용된다. 첫째, LLM 에 주입되는 카드에서는 예시 값을 제거하고 예시 값을 리터럴로 인용하지 말라는 주의를 덧붙인다. 예시 값이 있으면 소형 모델이 이를 실제 관측 값으로 오인하여 SQL 의 조건절에 사용하는 환각이 관찰되었기 때문이다. 다만 투영 매핑 노드는 원천 필드의 실재 여부를 카드 원문과 대조해야 하므로 예시 값을 포함한 원문을 받는다. 둘째, 컨텍스트의 각 요소에는 예산 상한이 있다. 로그 윈도우 80 행, 센서 분석 입력 16 건, 도구 결과 4,000 자, 소스 열람 120 행의 상한은 소형 로컬 모델의 컨텍스트 창 안에 가장 관련성이 높은 증거가 유지되도록 설계된 값이다. 그림은 두 소스의 컨텍스트가 push 와 pull 로 분석 그래프의 각 노드에 조합되는 과정을 나타낸 그림이다.

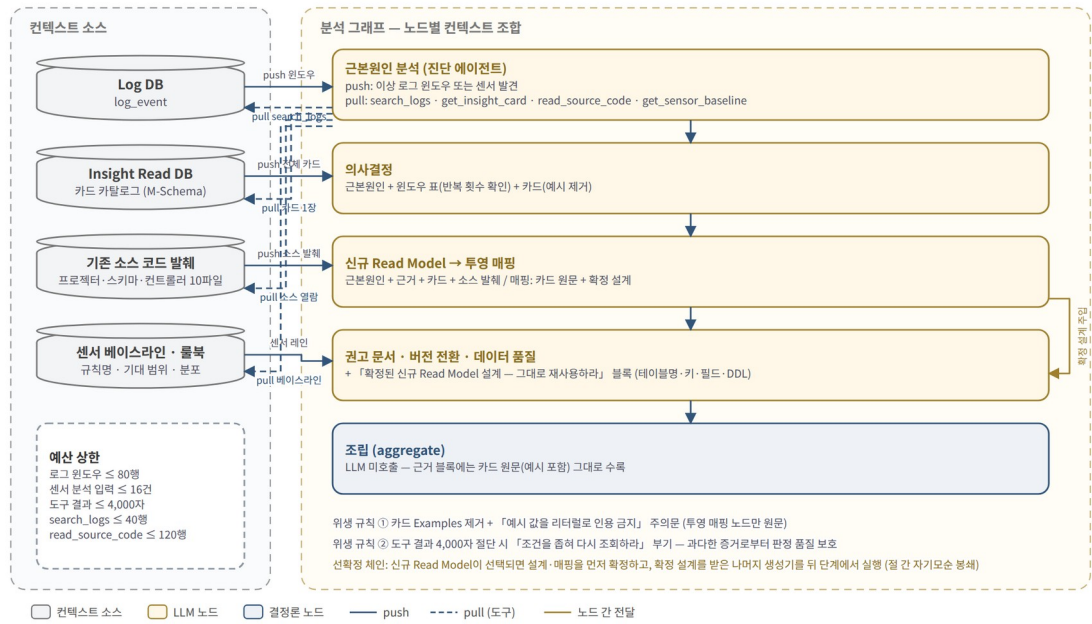


그림 3-6. 컨텍스트 조합 및 주입: push/pull 과 노드별 컨텍스트 구성

### 3.4 단일 산출물 Docs 설계

분석 그래프의 최종 산출물은 하나의 마크다운 문서인 Docs 이다. Docs 는 개발자가 읽고 곧 바로 Read Model 을 재생성할 수 있어야 한다는 요구와, 이 문서가 다시 소형 LLM 의 컨텍스트로 주입될 수 있어야 한다는 요구를 동시에 만족하도록 설계되었다. 후자 때문에 Docs 는 고정된 레이아웃과 토큰 예산을 갖는다. 문서는 상단의 YAML front-matter, 제목과 한 문장 결론(TL;DR), 근거 블록, 세 개의 필수 절, 선택 절, 최하단의 제약(Guardrails) 순으로 구성된다. 결론을 최상단에, 제약을 최하단에 두는 U 자형 배치는 긴 컨텍스트의 중간에 놓인 정보가 활용되지 못하는 현상[36]에 대응한 것이고, 근거 블록을 결론 바로 뒤이자 권고보다 앞에 고정한 것은 결론을 먼저 내리고 근거를 사후에 맞추는 사후 합리화를 구조적으로 막기 위한 배치이다. 그림은 Docs 의 레이아웃을 나타낸 그림이다.



그림 3-7. 단일 Docs의 고정 레이아웃

front-matter 는 문서를 기계적으로 식별하고 검증하기 위한 메타데이터를 담는다. 표는 front-matter 의 필드를 정리한 것이다.

| 필드                 | 내용                                                               |
|--------------------|------------------------------------------------------------------|
| docId, generatedAt | 문서 식별자(analysis-<상관관계 식별자> 또는 dq-<장면 키>)와 생성 시각                  |
| targetReadModel    | 대상 Read Model (권고 문서 → 데이터 품질 권고 → 버전 전환 → 신규 Read Model 순으로 결정) |
| sufficientEvidence | 세 절이 모두 근거로 채워졌는지 여부 (하나라도 센티넬이면 false)                          |
| apiVersion         | 버전 전환의 from·to 와 영향 받는 엔드포인트 목록 (버전 전환 코드에서 추출)                  |
| evidenceSources    | 본문이 인용하는 근거의 출처(developer-logging / insight-read-db)와 앵커 식별자 목록  |
| constraints        | 하단 Guardrails 에 재진술되는 6 개의 불변 제약                                 |

근거 블록은 두 컨텍스트 소스를 그대로 노출한다. <logging\_context>에는 이상 로그 윈도우가 빈도 요약과 표로, <insight\_read\_db>에는 대상 Read Model 의 카드가 M-Schema 로 실린다. 이어지는 세 필수 절이 연구 계획에서 정의한 Docs 의 3 요소이며, 근거가 부족하여 채울 수 없는 절은 INSUFFICIENT\_EVIDENCE 센티넬로 대체하여 형태를 유지한다. 6 개의 불변 제약은 v1 자산의 무손상, 기본 키 (scene\_key, attempt\_num)의 유지, TypeScript

의 any 금지, 식별자의 전체 단어 사용, DDL 실행과 API 컷오버의 인간 승인, Read Model 테이블명의 read\_ 접두 규칙이며 프로젝트의 불변 규칙과 1:1로 대응한다.

### 3.4.1 Read Model 재생성 권고 문서

권고 문서는 어떤 Read Model 을 왜 재생성 또는 보강해야 하는지에 대한 근거와 판단을 담으며, 아키텍처 의사결정 기록(Architecture Decision Record, ADR)[38]의 구조를 따라 Status, Context, Decision Drivers, Considered Options, Decision Outcome, Consequences, Non-Goals 의 소절로 구성된다. ADR 구조를 채택한 이유는 권고가 단일 결론이 아니라 기각된 대안과 트레이드오프를 함께 남겨야 개발자가 LLM 의 판단을 검토하고 다른 선택을 할 수 있기 때문이다. Context 에는 근거가 되는 로그 행이 수준, 행위명, 인용문, 해석의 형태로 나열되며 각 항목은 [corr:<상관관계 식별자>] 형식으로 출처를 명시한다. 이 인용 앵커는 front-matter 의 근거 출처 목록과 대조되어 실재하지 않는 로그를 근거로 삼는 환각을 사후 검증에서 잡아낼 수 있게 한다. Considered Options 에는 두 개 이상의 옵션이 접근 방식, 제안 필드, 트레이드오프, 코드 조각과 함께 제시되고, Decision Outcome 은 채택된 옵션과 이유를 서술한다.

권고 문서는 이상의 유형에 따라 세 형태로 특화된다. 신규 Read Model 이 함께 선택된 경우 Decision Outcome 은 확정된 설계의 채택을 서술하고 제안 필드는 그 설계의 필드와 일치한다. 적재 거절이나 poison 이벤트처럼 원천 데이터의 결함이 원인인 경우 권고 문서는 단독으로 생성되며 격리(containment) SQL 을 동반한다. 격리 SQL 은 결함 이벤트를 식별하는 조회, 오염된 Read Model 행을 무해화하는 갱신 또는 삭제, 결함 이벤트가 Event Store 에 유입되지 않았음을 검증하는 조회의 세 형태로 정형화되어 있으며, 프로젝션 커서를 임의로 전진시켜 결함 이벤트를 건너뛰는 SQL 은 정합성을 조용히 훼손하므로 생성이 금지된다. 센서 레인에서 발생한 데이터 품질 이상의 경우 권고는 ADR 대신 데이터 품질 권고의 형태를 취하여, 심각도와 근거, 기대 값과 실측 값의 차이를 장면 키·시도 번호·전역 시퀀스·영향 컬럼·관측 값·베이스라인 규칙으로 기술한 근거 목록, 영향 범위, 근본원인의 경로(상류 센서의 물리적 불가능, 투영 파이프라인 결함, 베이스라인 노후화), 격리·수정·강화(contain, fix, harden)의 단계별 옵션으로 구성된다.

### 3.4.2 Read Model 생성 SQL(DDL)

두 번째 절은 신규 Read Model 을 실제로 생성하는 데 필요한 산출물을 담는다. 절의 첫머리에는 이하의 변경이 인간의 승인 후에만 적용됨을 알리는 실행 게이트 문구가 고정된다. 본문은 대상 테이블명, 키 컬럼, 원천 이벤트를 밝힌 뒤 테이블과 인덱스를 생성하는 DDL 을 SQL 코드 블록으로 제시하고, 필드 목록을 Insight 카드와 동일한 M-Schema 튜플로 기술한다. 표는 신규 Read Model 생성기가 산출하는 구조화 출력의 필드와 각 필드에 적용되는 검증을 정리한 것이다.

| 필드                                   | 내용                                        | 검증                                   |
|--------------------------------------|-------------------------------------------|--------------------------------------|
| proposedName                         | read_ 접두의 스네이크 케이스 테이블명                   | 정규식, Drizzle 스키마의 테이블명과 일치           |
| purpose, rationale                   | 용도와 신규 설계의 근거 (앵커 로 그 인용)                 | —                                    |
| sourceEvents, keyColumns<br>fields[] | 원천 이벤트 유형과 키 컬럼<br>컬럼명(snake_case)·자료형·의미 | 키 컬럼이 fields에 실재<br>camelCase 자동 정규화 |
| migrationSql                         | CREATE TABLE + CREATE INDEX               | 실제 PostgreSQL에서 트랜잭션 실행 후 롤백         |
| drizzleSchema, projectorCode         | Drizzle 스키마와 Projector 구현                 | tsc --noEmit 컴파일, payload 키 실재       |
| controllerWiring                     | 스키마 export, 캐치업 메서드, 라우트, 모듈 등록           | 드리프트 키의 zod 확장 실재                    |

DDL 만으로는 빈 테이블이 만들어질 뿐이므로 이 절에는 테이블을 실제로 채우기 위한 두 부속 산출물이 동봉된다. 첫째는 투영 매핑 명세로, 원천 이벤트의 어떤 payload 필드가 어떤 컬럼으로 어떤 변환을 거쳐 반영되는지를 표로 기술하고, payload에서 유래하지 않는 파생 컬럼과 upsert 키, 리플레이 시 프로젝션 커서의 초기화에 관한 주의를 함께 담는다. 이 명세는 별도의 LLM 노드가 확정된 설계를 보고 생성하되, 대상 컬럼이 설계의 필드에 실재하고 원천 필드가 카드 원문에 실재하는 행만 남기는 근거 대조(grounding)를 거친다. 둘째는 Insight 카드 등록 SQL이다. 신규 Read Model의 구조화 출력(테이블명, 용도, 키 컬럼, 필드 목록)은 Insight 카드의 스키마와 동형이므로 LLM을 다시 호출하지 않고 insight\_entity와 insight\_field에 대한 INSERT문으로 결정론적으로 변환된다. 개발자가 DDL과 이 INSERT를 함께 적용하면 다음 분석부터 신규 Read Model이 카탈로그에 포함되며, 이것이 3.1 절의 페루프를 달는 구체적 장치이다. 신규 Read Model이 선택되지 않은 문서에서는 이

절에 격리 SQL 또는 위반 레코드의 식별·검증 SQL 이 실리고, 그마저 없으면 센티넬로 대체된다.

### 3.4.3 API Versioning 전략

세 번째 절은 Read Model 의 교체가 조회 API 에 미치는 변경을 다룬다. 제안 시스템의 버전 전략은 v1 자산의 무손상을 불변 규칙으로 삼는다. 신규 Read Model 은 기존 테이블을 변경하는 대신 \_v2 접미의 새 테이블로 만들어지고, 이를 채우는 프로젝트와 노출하는 라우트도 V2 접미와 /v2/ 경로의 새 자산으로 추가되며, 기존 파일의 수정은 모듈 등록이나 export 추가와 같은 한 줄 추가로 제한된다. 이는 블루-그린 배포[40]의 원리를 Read Model 수준에 적용한 것으로, Read Model 을 언제든지 재구성할 수 있다는 이벤트 소싱의 성질과 결합하여 v2 테이블을 이벤트 리플레이로 채운 뒤 검증하고 컷오버하며 문제가 있으면 v1 으로 즉시 되돌릴 수 있게 한다. 절의 본문은 Keep a Changelog 형식[41]을 따라 Unreleased 항목 아래 Added, Changed, Deprecated, Removed, Fixed, Security 의 범주로 변경을 기술하고, 하위 호환 변경과 파괴적 변경의 구분, 컷오버 전 테스트, 롤백 조건으로 구성된 마이그레이션 절차를 제시한다. 신규 Read Model 이 동반된 경우 절차에는 카드 등록 단계가 포함된다. 코드 변경은 파일 경로, 변경 종류(추가 또는 수정), 설명, 코드 조각의 목록으로 산출되며, 확정된 설계가 있을 때는 코드 조각의 라우트와 프로젝트 클래스명이 확정 설계와 일치하는지 검증되어 버전 전환 노드가 다른 이름의 사본을 만들어 문서가 분기하는 것을 막는다. 버전 전환이 선택되지 않은 문서에서는 API 버전에 미치는 영향의 서술로 대체된다.

### 3.4.4 Docs 저장 및 활용 워크플로

생성된 Docs 는 앵커 로그의 발생 시각과 문서 식별자를 조합한 파일명(<날짜>-<시각>-<docId>.md)으로 저장소의 llm-docs/ 디렉터리에 저장된다. 저장 직후 문서 계약 검증이 front-matter 의 존재, 세 필수 절의 고정 순서, Guardrails 의 존재, 근거 충분성 플래그와 센티넬의 정합, 본문의 인용 앵커가 근거 출처에 실재하는지, 선택 절을 제외한 코어 부분의 토크ン 예산(목표 8,000, 상한 20,000 토크ン)을 검사한다. 검증 결과는 로깅 규약에 따라 정보 수준으로만 기록되며 재실행을 유발하지 않는다.

Docs의 활용은 사람의 검토에서 시작된다. 개발자는 결론과 근거 블록으로 이상의 성격을 파악하고 권고 문서의 옵션과 트레이드오프를 검토하여 채택 여부를 결정한다. 채택하는 경우 두 번째 절의 DDL과 카드 등록 SQL을 실행하고, 선택 절의 프로젝터와 배선 코드를 저장소에 추가한 뒤 캐치업으로 신규 테이블을 이벤트 리플레이로 채우며, 세 번째 절의 절차에 따라 v2 라우트를 검증하고 컷오버한다. 이 과정에서 시스템은 세 가지 피드백 루프를 통해 자가 적응을 유지한다. 카탈로그 폐쇄 루프는 승인 후 등록된 카드가 다음 분석의 컨텍스트에 자동으로 편입되어 시스템이 새로 갖게 된 Read Model을 인지하는 경로이다. 카드 드리프트 감시 루프는 개발자가 DDL만 적용하고 카드 등록을 누락하면 관찰자가 이를 경고 신호로 방출하고 로그 레인이 카드 등록을 권고하는 Docs를 생성하여 누락을 회수한다. 자기 로그 발진 억제 루프는 파이프라인의 로그를 정보 수준으로 제한하는 규약으로 분석 결과가 새로운 분석을 유발하는 양의 피드백을 차단한다. 그림은 Docs의 저장에서 개발자의 검토와 승인, 적용, 그리고 다음 분석 사이클로 이어지는 순서와 카드 드리프트 감시의 개입을 나타낸 시퀀스 다이어그램이다.

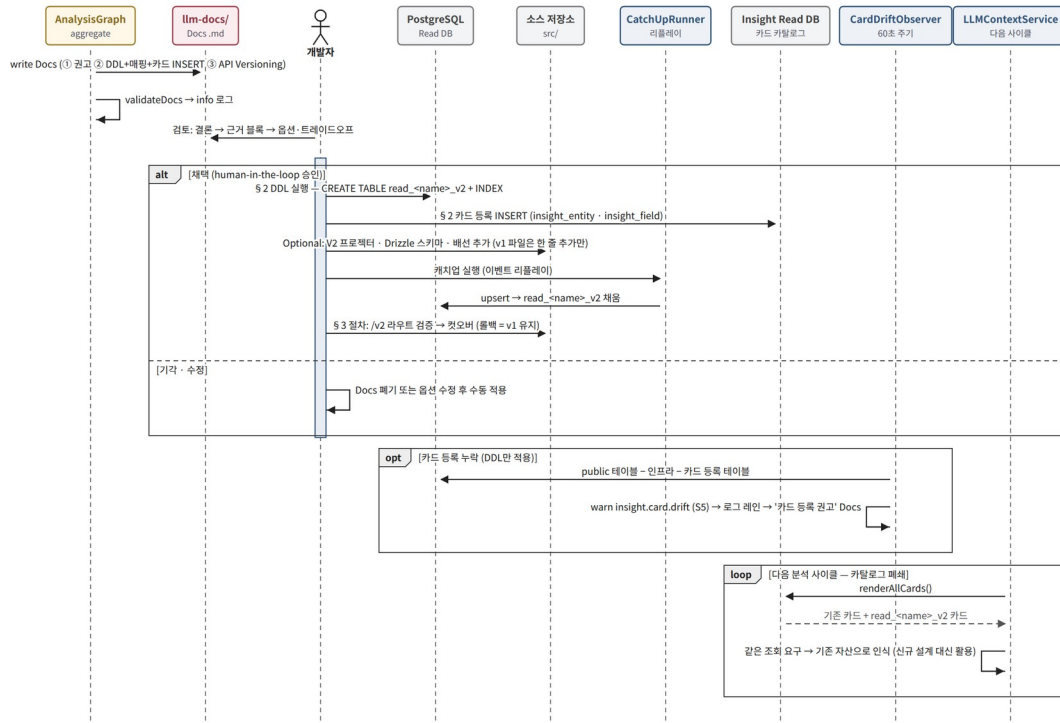


그림 3-8. Docs 승인·적용 시퀀스 다이어그램과 카탈로그 폐쇄 루프

### 3.5 LLM 파이프라인 구현

제안 시스템은 NestJS 기반의 단일 애플리케이션으로 구현되었으며, 이벤트 소싱·CQRS 영역, 관측 백본, LLM 영역이 기능 모듈로 분리되어 있다. 분석 그래프는 NestJS 모듈이 아닌 순수 라이브러리로 구현되어 로그 레인과 센서 레인의 두 모듈이 각자 진단 도구 구현체를 주입하여 인스턴스화한다. 표는 구현 환경을 정리한 것이다.

| 구분          | 구현                                                                           |
|-------------|------------------------------------------------------------------------------|
| 애플리케이션      | NestJS 11, TypeScript 5.9 (strict, any 금지), Drizzle ORM                      |
| 데이터베이스      | PostgreSQL 17 (TimescaleDB 2.17, log_event 는 1 일 청크의 hypertable)             |
| 메시지 브로커     | Apache Kafka 3 노드 KRaft 클러스터 (log-events, sensor-values), fluent-bit TCP 수집기 |
| 로깅          | Pino (nestjs-pino), 콘솔·파일·TCP 소켓 3 중 전송                                      |
| LLM 오케스트레이션 | LangGraph StateGraph[42], OpenAI 호환 채팅 클라이언트, zod 출력 스키마                     |
| LLM 런타임     | LM Studio 로컬 서버, 1 차 선별 qwen3.5-9b, 분석 qwen3.6-27b 계열, temperature 0         |

그림은 애플리케이션을 구성하는 NestJS 모듈과 라이브러리, 그리고 이들 사이의 의존 관계를 나타낸 컴포넌트 다이어그램이다. 관리 대상 시스템의 모듈과 관리 시스템의 모듈은 직접 호출로 결합되지 않고 Kafka 토픽과 로그 레일이라는 비동기 채널로만 연결되며, 두 레인 모듈은 각자 진단 도구 구현체를 주입하여 분석 그래프를 인스턴스화한다.

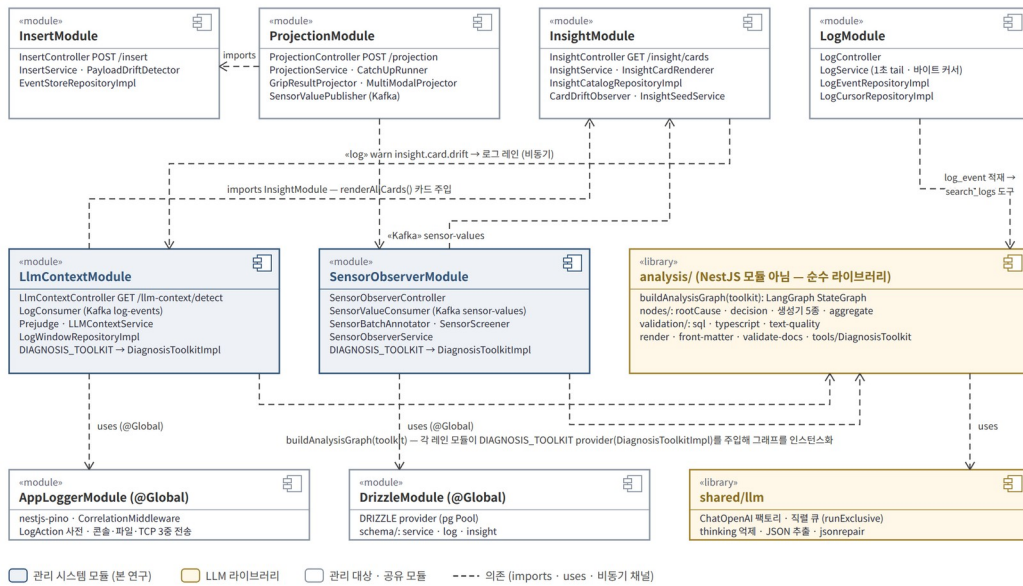


그림 3-9. NestJS 모듈 컴포넌트 다이어그램

**1차 선별.** 로그 레인의 소비자는 log-events 토픽을 5 초 주기로 폴링하여 최대 500 건의 배치를 만들되 정상 적재 로그와 상관관계 식별자가 없는 정보 수준의 프레임워크 로그를 제외한다. 배치에 경고 이상의 레코드가 하나라도 있으면 결정론적 프리게이트가 LLM 을 호출하지 않고 즉시 트립을 확정하고 해당 레코드의 상관관계 식별자를 트립 식별자로 반환한다. 이 프리게이트는 초기 구현에서 선판단 LLM 이 큰 배치 안의 경고 로그를 놓치는 미탐이 관찰되어 도입되었다. 경고 이상의 레코드가 없는 배치만 소형 LLM 의 선판단(prejudge)에 넘겨지며, 선판단 프롬프트는 하나의 상관관계 식별자가 요청 수신에서 정상 완료로 끝나는 궤적을 확실한 정상으로, 시작만 있고 완료가 없는 시퀀스 이상, 400 이상의 응답 코드, 요청 충족 실패, 동일 요청의 반복을 이상으로 정의하고 애매한 경우 트립으로 판정하는 fail-open 정책을 지시한다. 배치는 표로 압축하지 않고 원본 JSON 행 그대로 전달하여 맥락 필드의 손실을 막는다. 센서 레인은 sensor-values 토픽을 8 건 단위의 윈도우로 소비하며, 각 윈도우는 LLM 에 앞서 네 층의 결정론적 주석을 받는다. 물리(physical) 층은 파지 성공 여부의 정의역 위반, 회전 행렬의 직교성·행렬식 조건 위반, 0 이하의 파지 깊이, 이미지 범위를 벗어난 2D 포즈처럼 단독으로 불가능한 값을, 정합성(consistency) 층은 파지에 성공하였다고 기록된 시도의 이동 벡터와 깊이가 작업 공간을 벗어나는 모순을, 급변(jump) 층은 같은 장면 내에서 직전 시도 대비 평균 깊이가 0.1m 를 넘게 변하는 경우를, 통계(stat) 층은 관측 분포의 군집에서 robust-z

3.5를 넘어 벗어나는 값을 표시한다. 주석이 없는 윈도우는 LLM을 호출하지 않고 정상으로 확정되고, 주석이 있는 윈도우만 소형 LLM 스크리너가 수기 베이스라인 문서와 함께 해석하며, 물리·정합성 층의 확정 위반은 LLM의 판정과 무관하게 강제로 병합된다. 트립된 윈도우는 하나의 물리적 사건으로 누적되는 에피소드로 관리되어, 정상 윈도우가 이어지거나 15초간 입력이 없거나 64건에 도달하면 닫히면서 지목된 장면을 우선하여 16건 이내로 압축된 뒤 한번만 분석 그래프로 승급된다. 이 하이브리드 방식은 LLM 단독 판정 모드와 설정으로 전환할 수 있어 제 4장의 절제 실험(ablation)에 사용된다. 그림은 센서 값의 발행에서 4층 주석, 스크리너, 에피소드 누적을 거쳐 분석 그래프로 승급되는 센서 레인의 처리 순서를 나타낸 시퀀스 다이어그램이다.

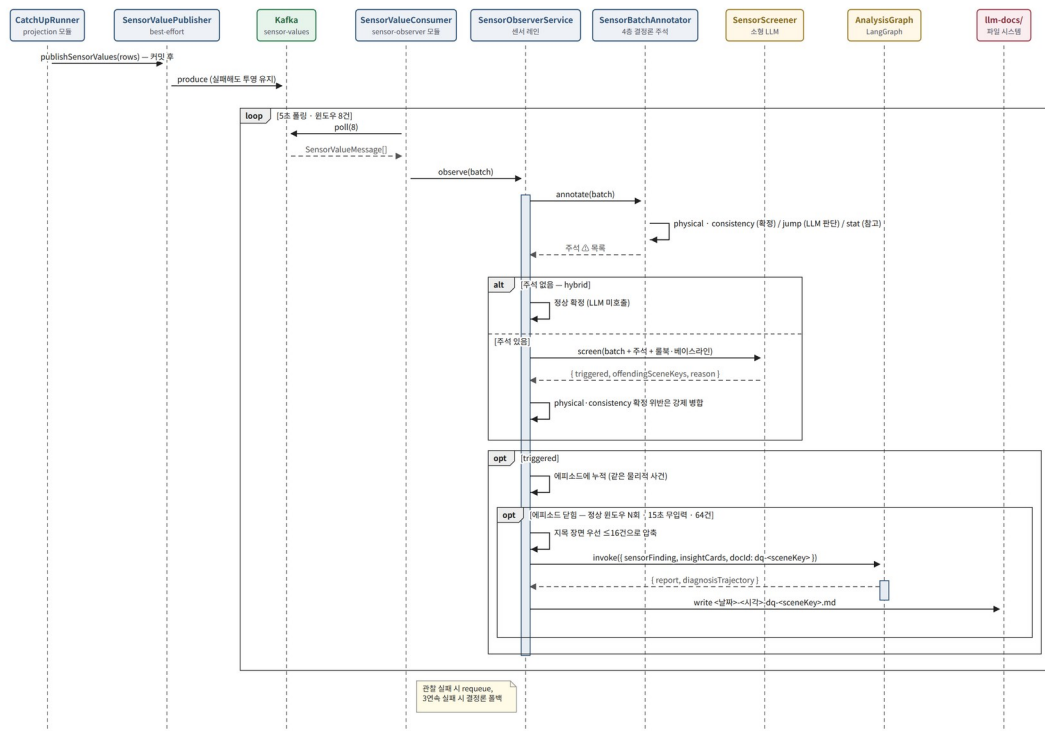


그림 3-10. 센서 레인 시퀀스 다이어그램 (발행 → 4층 주석 → 스크리너 → 에피소드 → 분석)

**분석 그래프.** 1차 선별을 통과한 입력은 LangGraph의 상태 그래프로 구현된 분석 그래프에 전달된다. 그래프의 상태는 입력 채널(이상 로그 윈도우 또는 센서 발견, Insight 카드), 진단 결과(근본원인, 도구 호출 궤적), 의사결정, 생성기별 출력, 최종 보고서로 구성되며, 생성기 출력 채널은 병렬 실행된 노드의 결과를 병합하는 리듀서를 갖는다. 그림은 분석 그래프의 실

행 흐름을 LLM 이 판단하는 구획과 결정론 코드가 처리하는 구획으로 나누어 나타낸 활동 다이어그램이다.

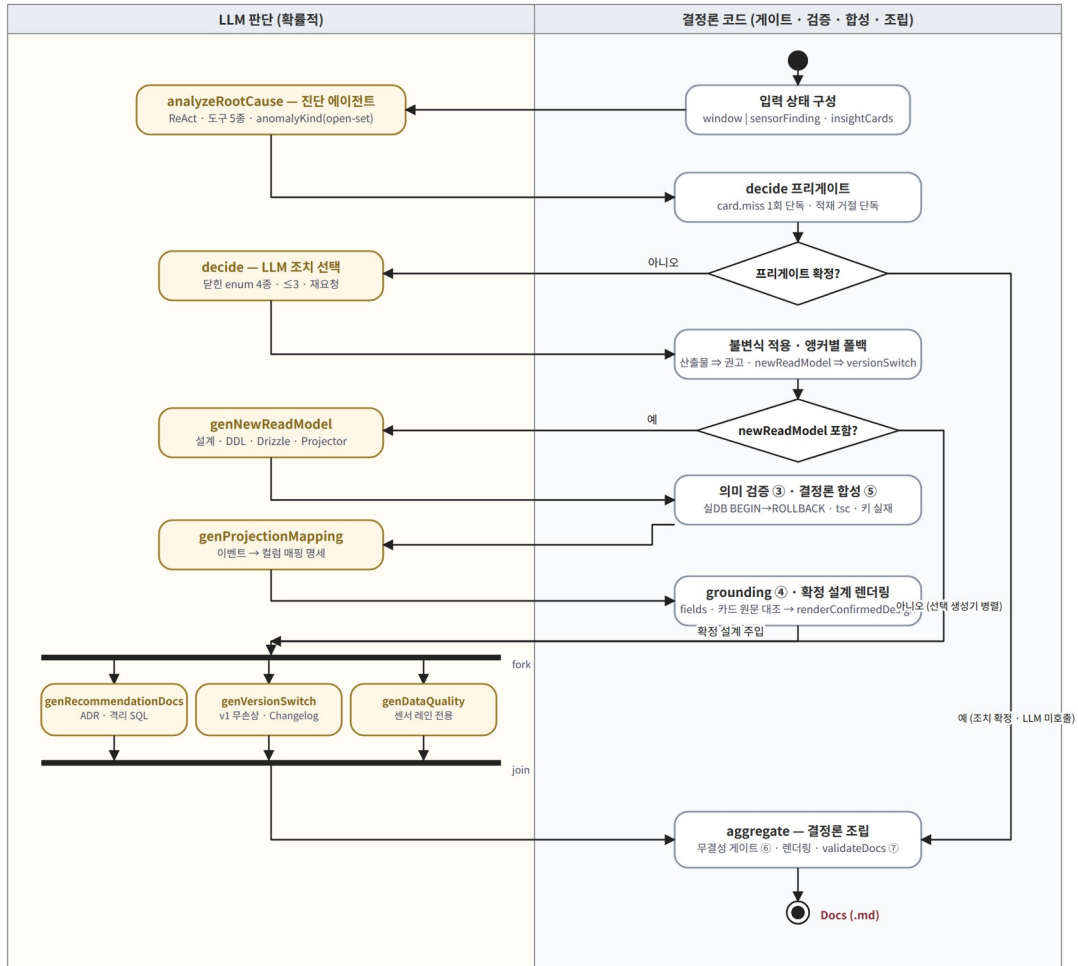


그림 3-11. 분석 그래프 활동 다이어그램 (LLM 판단/ 결정론 코드 구획)

그래프의 진입 노드는 근본원인을 분석하는 진단 에이전트이다. 진단 에이전트는 ReAct 방식 [25]으로 추론과 도구 호출을 반복하며 도구 왕복 4 회, 최종 출력 재요청 2 회, 전체 LLM 호출 8 회의 세 겹 상한을 갖고, 상한에 도달하면 지금까지의 증거로 최종 판정을 출력하라는 도구 메시지가 주입된다. 출력은 이상 유형(anomalyKind), 요약, 시간순 재구성, 실패한 요청 의도, 의심되는 Read Model 의 부족으로 구성되며 이상 유형은 열린 집합의 문자열이다. 프롬프트는 적재 단계의 검증 거절과 투영 단계의 변환 실패를 인과 경로 상에서 구분하고, 반복된 카드 조회 실패의 자연어 질의를 실패한 요청 의도로 그대로 옮기도록 지시한다. 그림은 진단

에이전트의 도구 호출 루프와, 도구 루프가 끝난 뒤 다른 노드와 공유하는 출력 검증·재요청 규약을 나타낸 시퀀스 다이어그램이다.

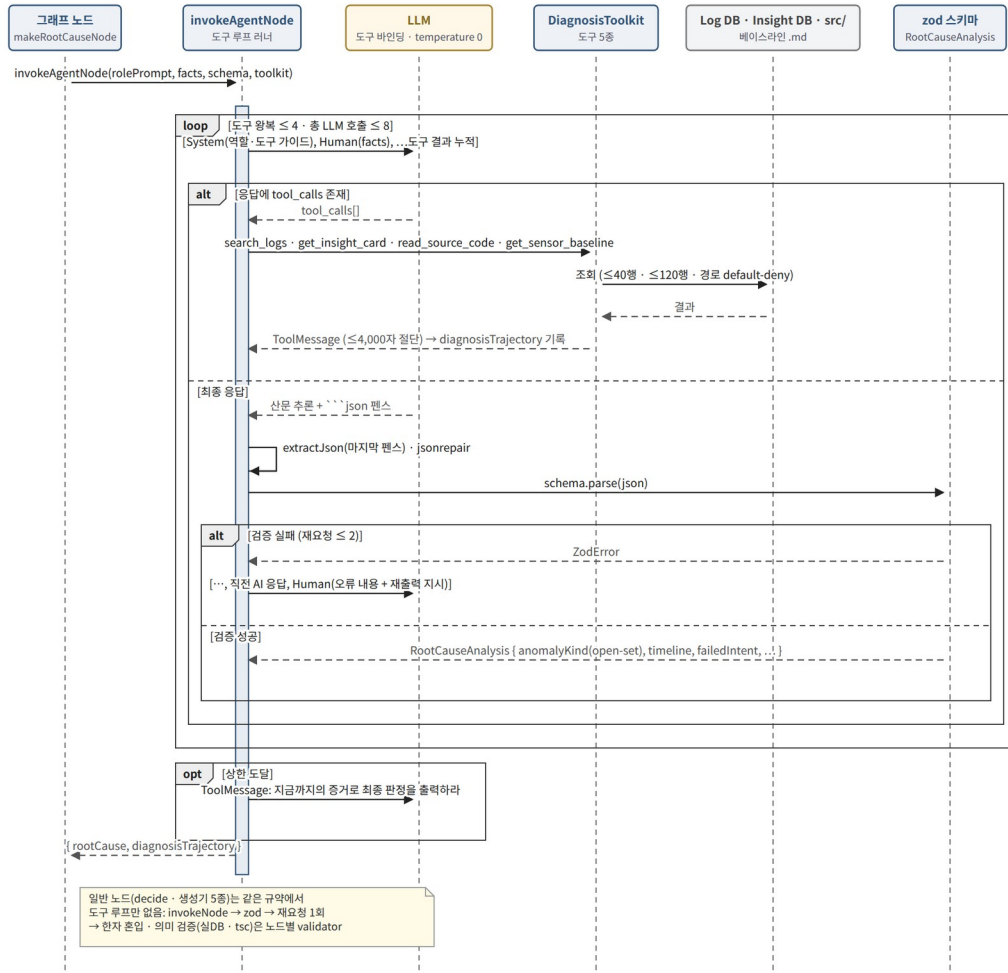


그림 3-12. 진단 에이전트 시퀀스 다이어그램 (도구 호출 루프와 출력 검증 규약)

의사결정 노드는 근본원인 분석의 결과로부터 네 조치의 닫힌 열거형에서 최대 세 개를 고르며, LLM의 선택 앞뒤로 결정론적 가드가 배치된다. 선택 이전의 두 프리게이트는 앵커가 카드 조회 실패 한 번뿐이고 다른 이상이 없으면 조회 의도가 드러나지 않았으므로 LLM을 호출하지 않고 조치 불필요로, 앵커가 적재 거절이면 결함 이벤트가 Event Store에 유입되지 않았으므로 격리 권고 단독으로 확정한다. 스키마 드리프트, poison 이벤트, 센서 발견, 두 번 이상 반복된 카드 조회 실패처럼 행동의 근거가 실재하는데 LLM이 빈 선택을 반환하면 모순을 지적하여 재요청하고, 그래도 비어 있으면 앵커별 기본 조치로 대체한다. 선택 이후의 두 불변식은 산출물(신규 Read Model 또는 버전 전환)이 선택되었는데 권고 계열이 없으면 권고 문서

를, 신규 Read Model 이 선택되면 버전 전환을 강제로 동반시킨다. 표는 앵커별 처리와 기본 조치를 정리한 것이다.

| 앵커 신호                     | 처리                | 확정 또는 기본 조치                       |
|---------------------------|-------------------|-----------------------------------|
| 카드 조회 실패 1 회 (단독)         | 프리게이트, LLM 미호출    | 조치 불필요                            |
| 적재 거절 (poison 없음)         | 프리게이트, LLM 미호출    | 권고 문서 (격리 SQL 동반)                 |
| poison 투영 실패              | LLM 선택 → 재요청 → 대체 | 권고 문서                             |
| 스키마 드리프트, 카드 조회 실패 2 회 이상 | LLM 선택 → 재요청 → 대체 | 신규 Read Model + 권고 문서 + 버전 전환     |
| 센서 발견                     | LLM 선택 → 재요청 → 대체 | 데이터 품질 권고 + 신규 Read Model + 버전 전환 |

생성기는 신규 Read Model, 투영 매핑, 권고 문서, 버전 전환, 데이터 품질 권고의 다섯 노드이며 라우팅은 두 단계로 나뉜다. 신규 Read Model 이 선택되면 그래프는 다른 생성기를 병렬로 실행하는 대신 신규 Read Model 노드와 투영 매핑 노드를 먼저 순서대로 실행하여 설계를 확정하고, 확정된 설계를 주입받은 나머지 생성기를 다음 단계에서 병렬로 실행한다. 신규 Read Model 이 선택되지 않았으면 선택된 생성기를 곧바로 병렬 실행한다. 이 선택정 체인은 3.3.3 항의 절 간 자기모순을 막기 위한 것으로, 병렬 실행만 사용하였던 초기 구현의 품질 감사에서 가장 빈번한 결함이 생성기마다 서로 다른 테이블명과 컬럼을 창안하는 것이었다는 결과에 근거한다. 조립 노드는 LLM 을 호출하지 않고 생성기 출력을 3.4 절의 레이아웃으로 렌더링하며, 산출물만 있고 권고가 없는 자기모순이 생성기의 실패로 발생하면 결정론적 대체 권고로 메우거나 산출물을 보류한다.

**프롬프트 구조.** 진단 에이전트를 제외한 모든 노드는 하나의 호출 규약을 공유한다. 시스템 메시지에는 노드의 역할, 판단 규칙, 출력 JSON 스키마를 담은 역할 프롬프트가, 사용자 메시지에는 3.3.3 항의 규칙으로 조립된 근거 텍스트가 놓인다. 모델은 자유로운 산문으로 추론한 뒤 마지막에 JSON 코드 블록을 출력하고 파이프라인은 마지막 코드 블록만을 추출하여 zod 스키마로 검증한다. 구조화 출력 API 대신 산문 추론 후 JSON 출력을 택한 것은 출력 형식을 문법으로 강제하는 것이 소형 모델의 추론 품질을 저해한다는 보고[39]에 근거한다. 표는 각 노드의 프롬프트 구성을 정리한 것이다.

| 노드      | 역할 프롬프트의 핵심 규칙 |              |              |
|---------|----------------|--------------|--------------|
|         | 직              | 근거 텍스트       | 출력           |
| 근본원인 분석 | 시간순 재구성, 실패 지점 | 원도우 또는 센서 발견 | anomalyKind, |

| 노드            | 역할 프롬프트의 핵심 규칙                                                                      | 근거 텍스트                                | 출력                                                      |
|---------------|-------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------|
|               | 의 인과 구분, 열린 집합 이상 유형 명명, 도구 우선 증거 규칙                                                | (+도구 결과 누적)                           | summary, timeline, failedIntent, suspectedReadModel Gap |
| 의사결정          | 4 개 조치의 정의, 신호별 규칙, 조치 불필요로 판정해야 하는 경우                                              | 근본원인 + 윈도우 표 + 카드                     | selected[], reasoning                                   |
| 신규 Read Model | read_ 접두 명명, payload 에서 유도 가능한 컬럼만, 집계 요구는 집계 차원을 키로, Projector 시그니처, 드립트 키의 zod 확장 | 근본원인 + 근거 + 카드 + 소스 발취                | 3.4.2 항의 출력 필드                                          |
| 투영 매핑         | 실재 필드만, 무변환은 verbatim, 파생 컬럼 분리, 리플레이 주의                                            | 근거 + 카드 원문 + 확정 설계                    | rows[], derivedColumns[], upsertKey, replayNote         |
| 권고 문서         | ADR 구조, [corr:] 인용, 격리 SQL 3 형, 커서 조작 금지                                            | 근본원인 + 의사결정 + 근거 + 카드 + 확정 설계 + 소스 발취 | ADR 필드, containmentSql, apiVersionImpact                |
| 버전 전환         | v1 무손상 규칙, Keep a Changelog 범주, 확정 설계의 클래스명·라우트 준수                                  | 근본원인 + 확정 설계 + 소스 발취                  | codeChanges[], changelogEntries[], 절차 필드                |
| 데이터 품질 권고     | 근거 우선 필드 순서, 열거형 값만 사용, contain·fix·harden 단계 규칙                                    | 센서 발견 + 근본원인 + 카드 + 베이스라인 + 확정 설계     | 심각도, sensorEvidence[], 옵션, containmentSql               |

**응답 검증.** LLM 의 응답이 Docs 에 실리기까지는 일곱 단계의 검증 관문을 거친다. 앞의 세 단계는 실패 시 직전 출력과 오류 내용을 대화에 누적하여 모델이 자신의 오류를 보고 수정하게 하는 재요청(re-ask)으로 되돌린다. temperature 0 에서 동일한 프롬프트의 재전송은 같은 실패를 결정론적으로 재현하므로 블라인드 재시도 대신 오류를 보여주는 재요청을 택한 것이다. 재요청은 1 회로 제한되며, 시간 초과와 같은 전송 실패는 별도의 예산(3 회)으로 관리되어 재요청 예산을 잠식하지 않는다. 표는 검증 관문을 정리한 것이다.

| 단계   | 검증         | 잡는 결함                                | 실패 시 |
|------|------------|--------------------------------------|------|
| ① 형태 | zod 스키마 파싱 | JSON 파싱 실패, 필드 누락, 자료형 불일치, 명명 규칙 위반 | 재요청  |

| 단계        | 검증                                                                                                      | 잡는 결함                                              | 실패 시                                |
|-----------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------------------|
| ② 산문 오염   | 한자(CJK) 혼입 검사                                                                                           | 로컬 모델의 한자 혼입<br>(정상 한국어에는 한자가<br>없어 오탐 없음)         | 재요청                                 |
| ③ 의미      | 실제 PostgreSQL 에서<br>트랜잭션 실행 후 롤백,<br>tsc --noEmit 컴파일, 드<br>리프트 키의 zod 확장<br>·payload 키·라우트·클래<br>스명 실재 | 실행되지 않는 SQL, 컴파<br>일되지 않는 코드,<br>payload 에 없는 키 접근 | 재요청, 소진 시 ⑤                         |
| ④ 근거 대조   | 관측 값의 근거 텍스트 실<br>재, 매핑 행의 대상 컬럼·<br>원천 필드 실재                                                           | 입력에 없는 값의 발명                                       | 행 제거, 근거 0 개면 출력<br>폐기              |
| ⑤ 결정론 합성  | 검증된 필드·키로부터<br>DDL, Drizzle 스키마,<br>Projector, 권고, 버전 전<br>환을 템플릿 합성                                    | 재요청 후에도 남은 컴파<br>일 불능 코드                           | 템플릿으로 교체, 유도 불<br>가 컬럼은 null + TODO |
| ⑥ 무결성 게이트 | 권고 없이 DDL·API 만<br>남은 자기모순                                                                              | 생성기의 부분 실패                                         | 대체 권고 합성, 불가 시<br>산출물 보류            |
| ⑦ 문서 계약   | front-matter, 3 절 순서,<br>Guardrails, 센티넬 정<br>합, 인용 앵커 실재, 토큰<br>예산                                     | 계약 위반                                              | 기록만 (재실행 없음)                        |

세 번째 단계의 의미 검증은 본 구현의 특징적인 부분이다. 생성된 DDL 은 실제 데이터베이스의 트랜잭션 안에서 실행된 뒤 롤백되어 문법이 아니라 실행 가능성이 검증되고, 생성된 프로젝트와 스키마 코드는 저장소 소스를 복사한 임시 디렉터리에 배치되어 실제 컴파일러로 검사되며 알려진 오류 유형에는 수정 힌트가 재요청 메시지에 병기된다. 이 단계가 잡는 것은 “그렇듯하지만 실행되지 않는 SQL”과 “payload 에 없는 키를 읽어 모든 컬럼이 null 이 되는 프로젝트”처럼 형태가 아니라 내용이 깨진 출력이며, 이는 LLM 은 설계 판단을 내리고 실행 가능성은 결정론적으로 보장한다는 3.1 절의 첫 번째 원칙을 구체화한 것이다. 그림은 검증 관문의 흐름과 실패 시의 재요청·합성 분기를 나타낸 활동 다이어그램이다.

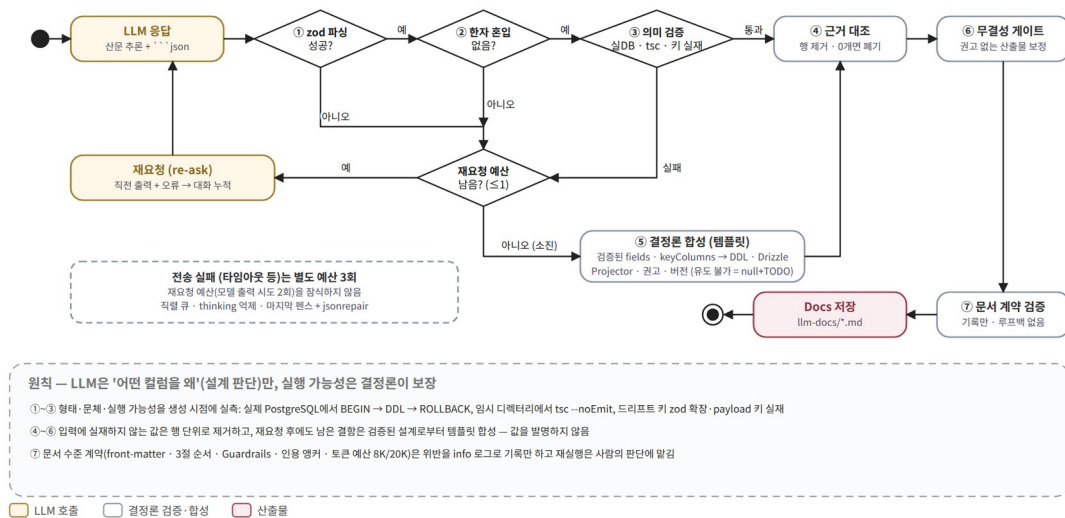


그림 3-13. LLM 응답 검증 활동 다이어그램

LLM 호출 인프라에는 로컬 모델 운용을 위한 세 가지 장치가 있다. 분석 그래프의 모든 LLM 호출은 프로세스 전역의 직렬 큐를 통과하는데, 그래프 수준의 병렬 실행이 단일 로컬 서버에 동시 요청을 보내면 서로의 시간 초과를 유발하는 연쇄가 관찰되었기 때문이다. 추론 과정을 출력하는 사고(thinking) 계열 모델에는 추론 출력을 억제하는 매개변수를 전달하여 출력 예산이 추론에 잠식되지 않도록 하고, JSON 추출은 응답의 마지막 코드 블록을 취하되 파싱 실패 시 복구 라이브러리로 교정을 시도한다. 이 장치들은 외부 API를 사용할 수 없는 환경에서 소형 로컬 모델로 파이프라인을 안정적으로 구동하기 위한 것이다.

### 3.6 MAPE-K 관점의 자가 적응 루프 분석

본 절에서는 제안 시스템을 MAPE-K 참조 모델[29]에 대응시켜 자가 적응 루프로서의 구조를 분석한다. MAPE-K는 관리 대상 시스템을 관측하는 Monitor, 적응의 필요성을 판단하는 Analyze, 대응 방안을 수립하는 Plan, 방안을 적용하는 Execute의 네 단계와 이들이 공유하는 지식 베이스로 구성된다. 표는 각 단계에 대응하는 제안 시스템의 구성 요소를 정리한 것이다.

| MAPE-K 단계 | 제안 시스템의 구성 요소                                                         | 판단 주체 |
|-----------|-----------------------------------------------------------------------|-------|
| Monitor   | 구조화 로깅 규약과 신호 카탈로그<br>(S1~S8), TCP·파일의 두 전송 경로, Log DB 적재, 센서 값 발행, 카 | 결정론   |

| MAPE-K 단계 | 제안 시스템의 구성 요소                                                                            | 판단 주체                               |
|-----------|------------------------------------------------------------------------------------------|-------------------------------------|
| Analyze   | 드 드리프트 관찰자<br>로그 레인의 필터·프리게이트·선폰<br>단, 센서 레인의 4 층 주석·스크리너<br>·에피소드, 진단 에이전트의 근본원<br>인 분석 | 결정론 + 소형 LLM, 진단 에이전트<br>(LLM + 도구) |
| Plan      | 의사결정 노드(프리게이트·LLM 선<br>택·불변식), 생성기 5 종, 검증 관문<br>①~⑥, Docs 조립                            | LLM(설계 판단) + 결정론(검증·합<br>성·조립)      |
| Execute   | Docs 저장과 계약 검증, 인간 승인,<br>DDL 실행·프로젝터 배선·리플레이<br>·카드 등록·API 컷오버                          | 사람 (human-in-the-loop)              |
| Knowledge | Insight Read DB(카드 카탈로그),<br>Log DB(이력), 센서 베이스라인·물<br>북, 기존 소스 코드 발췌                    | —                                   |

Monitor 단계는 전적으로 결정론적이다. 관리 대상 시스템은 관리 시스템을 위한 별도의 센서를 갖지 않으며 로깅 규약을 따르는 구조화 로그와 센서 값 토픽이 센서의 역할을 한다. 이는 센서와 실행기를 관리 대상 시스템에 부착하는 통상의 방식[12]과 비교할 때 기존 시스템의 관측 수단을 그대로 센서로 재사용하여 관리 대상 시스템의 변경을 최소화한다. Analyze 단계는 두 층으로 나뉜다. 1 차 선별은 모든 배치에 실행되므로 결정론적 게이트와 소형 LLM 으로 비용을 제한하고, 근본원인 분석은 선별을 통과한 소수의 신호에 대해서만 도구 호출을 동반한 에이전트로 수행된다. 전통적인 접근에서 Analyze 가 규칙이나 모델에 대한 위반 탐지로 구현되는 것과 달리 제안 시스템의 Analyze 는 이상의 유형을 열린 집합으로 명명하고 근본원인을 자연어로 설명한다.

Plan 단계는 제안 시스템이 전통적인 자가 적응 기법과 가장 크게 구별되는 지점이다.

Rainbow 와 같은 기존 접근[30]에서 Plan 은 사전에 정의된 적응 전략 가운데 하나를 선택하는 것이지만, 제안 시스템의 Plan 은 신규 Read Model 의 스키마, DDL, 프로젝트 코드, API 버전 전환 절차처럼 설계 시점에 열거할 수 없는 산출물을 실행 시점에 생성한다. 다만 어떤 종류의 산출물을 만들 것인가의 선택은 네 조치의 닫힌 열거형으로 제한되므로, 제안 시스템의 적응 공간은 조치의 종류에서는 닫혀 있고 조치의 내용에서는 열려 있는 이중 구조를 갖는다. 이 구조 안에서 LLM 의 확률적 판단은 프리게이트, 불변식, 일곱 단계의 검증 관문으로 둘러싸여 있으며 검증을 통과하지 못한 판단은 결정론적 합성으로 대체되거나 보류된다.

Execute 단계는 자동화되지 않는다. 제안 시스템은 생성된 DDL 과 코드를 스스로 적용하지 않고 Docs 로 산출하는 데서 멈추며, 적용은 사람이 문서를 검토하고 승인한 뒤에 수행한다. 이것이 제안 시스템의 human-in-the-loop 개입 지점이며 그 위치는 Plan 과 Execute 사이의 경계이다. 이 지점을 사람에게 남긴 이유는 두 가지이다. LLM 이 생성한 스키마는 문법적으로 유효하고 실행 가능하더라도 도메인의 관점에서 부적절할 수 있으며[24] 이를 판단하는 최종 권한은 도메인을 아는 사람에게 있어야 한다는 점, 그리고 DDL 의 적용과 API 컷오버는 되돌리기 어려운 변경이므로 자동 적용 시 LLM 판단의 오류가 데이터 계층으로 전파된다는 점이다. 제안 시스템은 이 위험을 v1 무손상의 버전 전략과 인간 승인 게이트의 두 장치로 통제한다. 사람이 개입하는 두 번째 지점은 Knowledge 의 갱신이다. 신규 Read Model 의 카드는 사람이 DDL 과 함께 등록하며 시스템은 카드 드리프트 감시로 누락을 회수할 뿐 카탈로그를 스스로 갱신하지 않는다. 그림은 제안 시스템을 MAPE-K 루프에 대응시킨 그림이다.

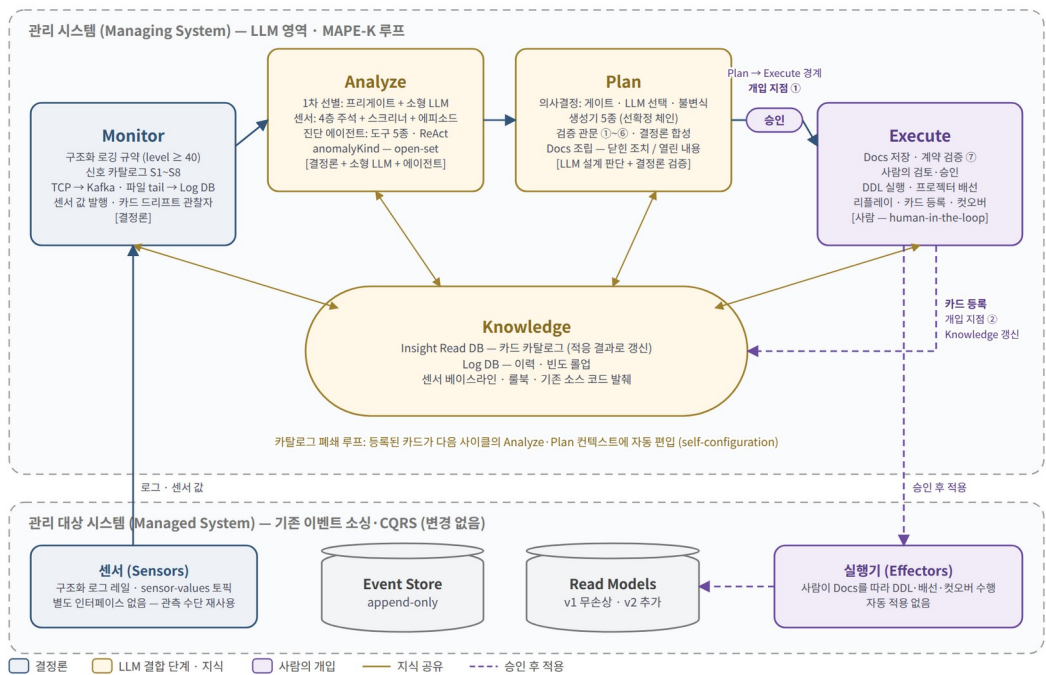


그림 3-14. MAPE-K 관점에서 본 Self-Adaptive CQRS 의 자가 적응 루프와 Human-in-the-Loop 개입 지점

Knowledge 는 제안 시스템에서 자가 적응을 반복 가능하게 만드는 요소이다. 전통적인 접근에서 지식 베이스가 아키텍처 모델과 정책으로 구성되어 설계 시점에 대부분 확정되는 것과 달리 제안 시스템의 Insight Read DB 는 적응의 결과로 갱신된다. 하나의 루프에서 생성된 신규

Read Model의 카드가 등록되면 다음 루프의 Analyze와 Plan은 이를 기존 자산으로 인식하여 같은 조회 요구에 대해 신규 설계 대신 기존 모델의 활용을 판단하게 된다. 이 카탈로그 페쇄 루프는 자율 컴퓨팅이 제시한 자기 구성(self-configuration)의 성질[9]을 Read Model 수명주기의 생성·진화 단계에서 구현한 것으로 볼 수 있다. 반면 루프의 안정성 측면에서 제안 시스템은 두 가지 한계를 갖는다. 문서 계약 검증에 실패한 Docs는 기록될 뿐 재실행되지 않으며, Execute 단계의 결과, 즉 개발자가 권고를 채택하였는지와 적용 후 조회 요구가 실제로 충족되었는지는 카드 등록의 유무 외에는 루프에 되먹임되지 않는다. 이 두 한계는 제 5 장에서 향후 연구로 다룬다.

---

## 참고 문헌 (3 장 신규 추가분 — 2 장 [37]에 이어짐)

- [36] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. ※ 2 장에서 미인용 상태였으나 3.4 절(Docs U 자형 배치)에서 인용 — 삭제하지 않고 유지
- [38] M. Nygard, “Documenting Architecture Decisions,” 2011. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [39] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models,” arXiv preprint arXiv:2408.02442, 2024.
- [40] M. Fowler, “BlueGreenDeployment,” martinowler.com, 2010. [Online]. Available: <https://martinfowler.com/bliki/BlueGreenDeployment.html>
- [41] O. Lacasse, “Keep a Changelog,” v1.1.0. [Online]. Available: <https://keepachangelog.com/>
- [42] LangChain, “LangGraph Documentation,” 2025. [Online]. Available: <https://langchain-ai.github.io/langgraph/>